Tecnológico de Costa Rica

Escuela de Ingeniería en Computadores

Seguridad de la Información • CE-1115

Taller de Criptografía Aplicada

Chat cifrado entre dos procesos locales con criptografía híbrida

SecureChat: descripción del protocolo

¿Qué viaja por el socket y qué nunca sale de cada extremo?

RSA-OAEP • AES-256-GCM • Handshake • Nonce • Authentication tag

Repositorio: `https://github.com/Adriel23456/SecureChat_Workshop`

PROFESOR

MSc. Andrés F. Vargas Rivera

INTEGRANTES — TRABAJO EN PAREJAS

Nombre	Carné
Adriel S. Chaves Salazar	2021031465
Henry D. Nuñez Pérez	2022089224

II Semestre, 2026 • Octubre de 2026 • Cartago, Costa Rica

Índice

1 Introducción y objetivo **1**
- 1.1 Alcance . . . 1
- 1.2 Entorno de desarrollo . . . 1

2 Arquitectura general **1**
- 2.1 Organización del código . . . 2
- 2.2 Concurrencia . . . 3

3 Material criptográfico **3**
- 3.1 Par de llaves asimétricas . . . 3
- 3.2 Llave pública y llave privada: qué hace cada una . . . 3
- 3.3 Huella de la llave pública . . . 4
- 3.4 Distribución de las llaves públicas . . . 4
- 3.5 Uso del par de llaves de cada participante . . . 5
- 3.6 Clave de sesión . . . 5
- 3.7 Qué es público, qué es secreto y qué viaja por la red . . . 5

4 Establecimiento de la sesión (handshake) **6**
- 4.1 Mensajes del handshake . . . 6
- 4.2 Generación y protección de la clave de sesión . . . 7
- 4.3 Transcript, firma y confirmación . . . 7
- 4.4 Autenticación mutua . . . 7
- 4.5 Manejo de errores . . . 8

5 Protección de los mensajes **8**
- 5.1 AES-256-GCM . . . 9
- 5.2 Generación del nonce . . . 9
- 5.3 Verificación en el receptor . . . 9
- 5.4 Protección contra repetición . . . 10
- 5.5 Evidencia en pantalla . . . 10

6 Formato de los mensajes transmitidos **10**

7 Conversación bidireccional y cierre de sesión **11**
- 7.1 Concurrencia y consola . . . 11
- 7.2 Presentación en consola . . . 12
- 7.3 Comandos locales . . . 12
- 7.4 Cierre controlado . . . 12

8 Detección de manipulación **13**
- 8.1 Mecanismo de la prueba controlada . . . 13
- 8.2 Resultado . . . 14
- 8.3 Política después de un rechazo . . . 15
- 8.4 Pruebas automatizadas . . . 15

9 Análisis de seguridad **15**
9.1 Modelo de adversario 15
9.2 Cumplimiento de los requisitos mínimos 16
9.3 Limitaciones conocidas 17

10 Preguntas de cierre **17**

11 Evidencias de la demostración **18**

Referencias **20**

A Ejecución y estructura del repositorio **21**

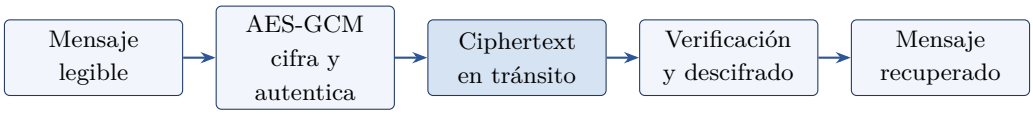
B Declaración de uso de IA **21**

1. Introducción y objetivo

SecureChat es una aplicación de consola que mantiene una conversación bidireccional entre dos procesos que se ejecutan en la misma computadora, conectados por un socket TCP sobre `localhost`. La criptografía **asimétrica** se usa para establecer la sesión y la criptografía **simétrica autenticada** protege cada mensaje.

La solución está escrita en Python 3 con la biblioteca `cryptography` (Python Cryptographic Authority, 2026). Cada participante tiene un par de llaves RSA-3072. Durante el handshake, Alice genera una clave de sesión aleatoria de 256 bits y la envía a Bob cifrada con RSA-OAEP (Moriarty et al., 2016). A partir de ese momento, cada mensaje se protege con AES-256-GCM (Dworkin, 2007): por el socket solo viajan el nonce, el ciphertext y el tag de autenticación, nunca el texto escrito por el usuario. En la demostración, ambas instancias completaron el handshake, intercambiaron más de tres mensajes por sentido y rechazaron todos los mensajes alterados a propósito, sin mostrar su contenido.

El flujo de cada mensaje es el que pide el enunciado:



1.1. Alcance

- **Cubre:** dos participantes fijos (Alice y Bob) en la misma computadora, generación y custodia local de las llaves RSA, autenticación mutua por huellas verificadas fuera de banda, establecimiento de una clave de sesión nueva por conexión, cifrado autenticado de cada mensaje, protección contra repetición y una prueba controlada de manipulación.
- **No cubre:** más de dos participantes, comunicación entre computadoras distintas, persistencia de los mensajes, revocación de llaves ni secreto hacia adelante (apartado 9.3).

1.2. Entorno de desarrollo

Tabla 1.1: Entorno de desarrollo y demostración

Elemento	Detalle
Sistema operativo	Windows 10/11 (PowerShell, Windows Terminal y terminal de VS Code)
Lenguaje	Python 3.10 o superior, 64 bits (probado con Python 3.15)
Biblioteca criptográfica	`cryptography` (`rsa`, `padding`, `AESGCM`, `hashes`, `serialization`)
Pruebas	`pytest`, 41 pruebas automáticas
Comunicación	Sockets TCP/IP sobre `127.0.0.1`, puerto 5050
Entornos virtuales	Uno por participante: `srcAlice\.venv` y `srcBob\.venv`
Repositorio	`https://github.com/Adriel23456/SecureChat_Workshop`

2. Arquitectura general

Cada participante ejecuta su propio programa desde su carpeta (figura 2.1). Bob espera conexiones (servidor) y Alice inicia la conexión (cliente). El código que ambos comparten vive en `srcShared`.

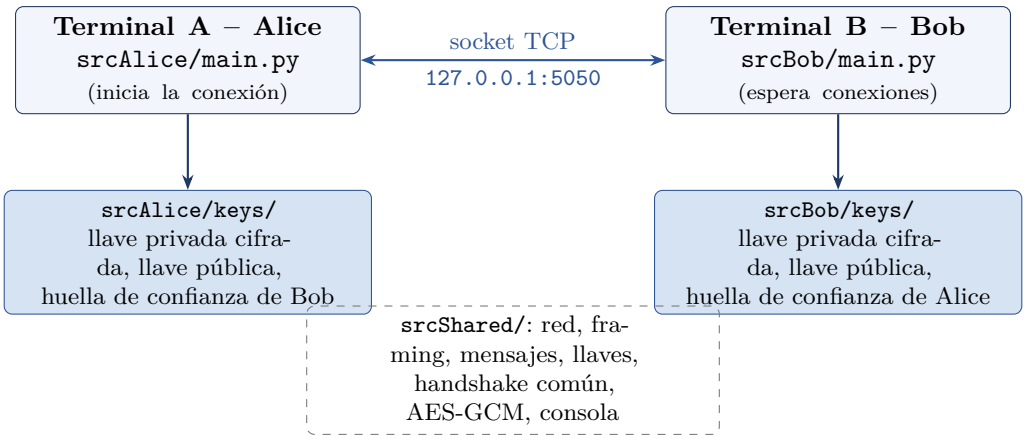


Figura 2.1: Arquitectura de SecureChat: dos procesos conectados por TCP en la misma computadora. La clave de sesión solo existe en la memoria de ambos procesos durante la conversación.

2.1. Organización del código

El diseño sigue programación orientada a objetos y los principios SOLID: cada clase tiene una sola responsabilidad y las dependencias se inyectan desde `main.py`. Todas las constantes están en `config.py` (de cada participante) o en `srcShared/shared_config.py` (las del protocolo, que deben coincidir en ambos lados).

Tabla 2.1: Módulos principales y su responsabilidad

Módulo	Responsabilidad
`srcBob/tcp_server.py`, `srcAlice/tcp_client.py`	Abrir la conexión TCP (`accept` o `connect`).
`framed_connection.py`	Enviar y recibir tramas completas con prefijo de longitud.
`message.py`	Tipos de mensaje y conversión mensaje ↔ bytes.
`key_store.py`, `rsa_key_generator.py`, `key_fingerprint.py`, `trusted_peer_store.py`	Generar, guardar y cargar llaves; calcular y registrar huellas.
`handshake_base.py`, `alice_handshake.py`, `bob_handshake.py`	Handshake común y pasos propios de cada rol.
`rsa_operations.py`, `aes_gcm_cipher.py`	Envoltorios de RSA-OAEP, RSA-PSS y AES-GCM.
`nonce_sequence.py`, `secure_channel.py`	Nonces por dirección, cifrado y verificación de cada registro.
`chat_session.py`, `chat_commands.py`	Conversación: hilos, comandos y cierre.
`record_tamperer.py`, `rejection_policy.py`	Prueba controlada de manipulación y política tras un rechazo.
`console_view.py`, `keyboard_input.py`	Todo lo que se muestra y se lee en la consola.

2.2. Concurrencia

Durante el chat, cada instancia usa dos hilos: el principal lee el teclado y envía; un hilo receptor espera tramas, las verifica y las muestra. En Windows, `select()` solo funciona con sockets y no con `stdin`, por lo que no se puede esperar a la vez el teclado y la red con un solo hilo. Los detalles de la consola se describen en apartado 7.

3. Material criptográfico

Cada participante dispone de su propio par de llaves RSA, generado y guardado dentro de su carpeta (`srcAlice/keys/` y `srcBob/keys/`). Ninguna de estas carpetas se versiona: la regla `keys/` del `.gitignore` las excluye del repositorio.

3.1. Par de llaves asimétricas

Tabla 3.1: Parámetros del par de llaves asimétricas

Parámetro	**Valor**
Algoritmo	RSA
Tamaño del módulo	3072 bits (se rechaza al cargar cualquier llave menor a 2048 bits)
Exponente público	$e = 65537$
Generación	`rsa.generate_private_key` de `cryptography`, con el generador aleatorio seguro del sistema operativo
Relleno para cifrar	OAEP con SHA-256 y MGF1(SHA-256) (Moriarty et al., 2016)
Relleno para firmar	PSS con SHA-256 y MGF1(SHA-256)
Formato de la llave privada	PEM, PKCS#8, **cifrada** con una frase de paso (`BestAvailableEncryption`)
Formato de la llave pública	PEM, *SubjectPublicKeyInfo* (X.509); 422 bytes en DER
Frase de paso	Mínimo 8 caracteres; se pide sin eco en pantalla (`getpass`) y nunca se guarda
Archivos	`keys/private_key.pem` y `keys/public_key.pem` dentro de la carpeta de cada participante

Se eligió RSA-3072 en lugar del mínimo de 2048 bits porque ofrece alrededor de 128 bits de seguridad (Barker, 2020), un nivel que se mantiene vigente a largo plazo. El costo es una generación de llaves algo más lenta (uno o dos segundos), que ocurre una sola vez. El exponente $e = 65537$ es el valor estándar. Se usa OAEP y no el relleno PKCS#1 v1.5, que es vulnerable a ataques de texto cifrado escogido (Bleichenbacher, 1998).

3.2. Llave pública y llave privada: qué hace cada una

Las dos llaves están relacionadas matemáticamente: lo que una hace, solo la otra lo deshace. La llave pública (n, e) se deriva de la privada, pero el camino inverso exige factorizar n, lo cual es inviable para un módulo de 3072 bits. Por eso:

- **La llave pública puede compartirse.** Sirve para *cifrar hacia* su dueño y para *verificar* sus firmas. Conocerla no permite descifrar ni firmar.

- **La llave privada debe permanecer bajo control exclusivo de su dueño.** Es la única que puede *descifrar* lo cifrado con la pública y *firmar*. Nunca se muestra en pantalla, nunca se escribe sin cifrar y nunca viaja por el socket.
- **La llave pública no reemplaza a la privada.** Con la pública de Bob, Alice puede proteger la clave de sesión para que solo Bob la lea, pero no puede leerla ella misma una vez cifrada ni hacerse pasar por Bob. Si la pública bastara para descifrar, cualquiera que la conociera (es decir, todos) podría hacerlo.

Prueba de correspondencia. La opción *Verificar mis llaves* del menú carga ambos archivos y ejecuta tres comprobaciones: (1) el módulo n y el exponente e derivados de la privada coinciden con los de la pública; (2) una firma PSS hecha con la privada es aceptada por la pública; y (3) un valor aleatorio cifrado con OAEP usando la pública se recupera con la privada. El valor de prueba es aleatorio, se descarta y solo se informa "aprobado" o "fallido".

3.3. Huella de la llave pública

La huella es el resumen SHA-256 de la codificación DER de la llave pública: 32 bytes que se muestran como `SHA256:ab:cd:...`. Una llave de 3072 bits ocupa cientos de bytes y no se puede comparar a simple vista; la huella sí. Además, si un atacante sustituye la llave pública durante el handshake, la huella cambia por completo. La huella es información pública: mostrarla no revela ningún secreto. Es lo único de las llaves que el programa muestra en pantalla.

3.4. Distribución de las llaves públicas

Decisión. Se adoptó un modelo mixto. La llave pública viaja por el socket durante el handshake, pero antes de usarla el receptor compara su huella SHA-256 con una huella de confianza registrada previamente. Para registrarla, cada participante muestra su huella en su terminal (opción 3 del menú) y el otro la registra (opción 4) después de compararla por un canal independiente de la red (*fuera de banda*). Si las huellas no coinciden, el handshake se aborta. Así se cierra la puerta al atacante en el medio que sustituye la llave pública.

Las huellas de confianza se guardan en `srcBob/keys/trusted_alice_fingerprint.txt` y `srcAlice/keys/trusted_bob_fingerprint.txt`.

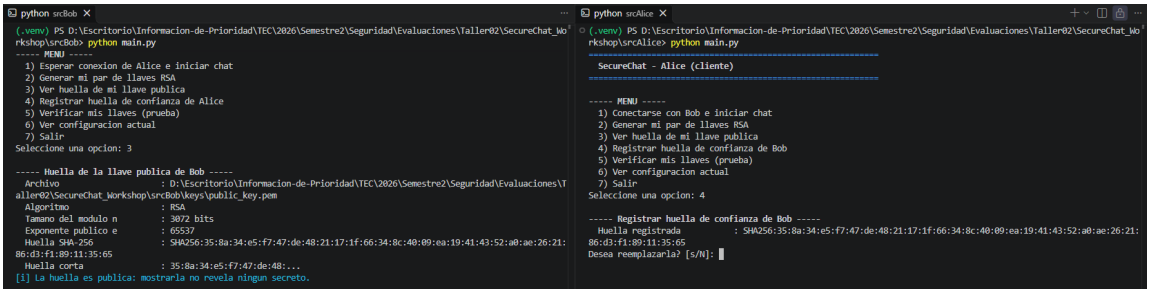


Figura 3.1: Opción 3 (huella propia) en una terminal y opción 4 (registro de la huella del par) en la otra.

3.5. Uso del par de llaves de cada participante

- **Par de Bob:** Alice cifra la clave de sesión con la llave pública de Bob (RSA-OAEP) y solo Bob puede recuperarla con su privada. Que Bob logre confirmar la sesión con esa clave demuestra que posee la privada correspondiente a la huella registrada.
- **Par de Alice:** Alice firma su mensaje del handshake con su llave privada (RSA-PSS) y Bob verifica la firma con la llave pública de Alice, previamente autenticada por su huella. Así Bob sabe que la clave de sesión la generó Alice y no un tercero.

Con este esquema la autenticación es **mutua** y cada par de llaves cumple una función concreta.

3.6. Clave de sesión

La clave de sesión K tiene 256 bits. Alice la genera en cada conexión con el generador aleatorio criptográfico del sistema operativo (función `generate_key` de `AESGCM`). No está escrita en el código ni se guarda en disco: vive solo en la memoria de los dos procesos durante la sesión. Viaja una única vez, cifrada con RSA-OAEP, y al cerrar la sesión se eliminan todas las referencias a ella (apartado 7.4). Cada conexión nueva produce una clave distinta.

3.7. Qué es público, qué es secreto y qué viaja por la red

Tabla 3.2: Clasificación del material y de los datos del protocolo

Elemento	Dónde vive	¿Viaja por el socket?	Nivel
Llave privada (Alice, Bob)	`keys/private_key.pem`, cifrada; en memoria solo mientras se usa	Nunca	**Secreto**
Frase de paso	Solo en la memoria de su dueño	Nunca	**Secreto**
Llave pública	`keys/public_key.pem`	Sí, en el handshake	**Público**
Huella de la llave pública	Pantalla y archivo de confianza del par	No (se compara fuera de banda)	**Público**
Clave de sesión	Memoria de ambos procesos	Solo cifrada con RSA-OAEP	**Secreto**
Plaintext del usuario	Solo en la terminal local	Nunca (tras el handshake)	**Secreto**
Valores aleatorios del HELLO	Generados por sesión	Sí	**Público**
Nonce	Generado por mensaje	Sí, junto al ciphertext	**Público**
Ciphertext	En tránsito	Sí	**Público**
Authentication tag	En tránsito	Sí	**Público**
Metadatos (puertos, tamaños, horas)	Encabezados TCP/IP y prefijo de longitud	Sí	**Público**

4. Establecimiento de la sesión (handshake)

Antes de permitir cualquier mensaje de usuario, ambas instancias completan un handshake de seis mensajes (figura 4.1). La criptografía asimétrica cumple aquí dos funciones: **proteger** la clave de sesión (RSA-OAEP con la llave pública de Bob) y **autenticar** a los participantes (huellas verificadas fuera de banda y firma RSA-PSS de Alice). Al terminar, ambos poseen la misma clave de sesión K de 256 bits sin que esta haya viajado nunca en texto plano.

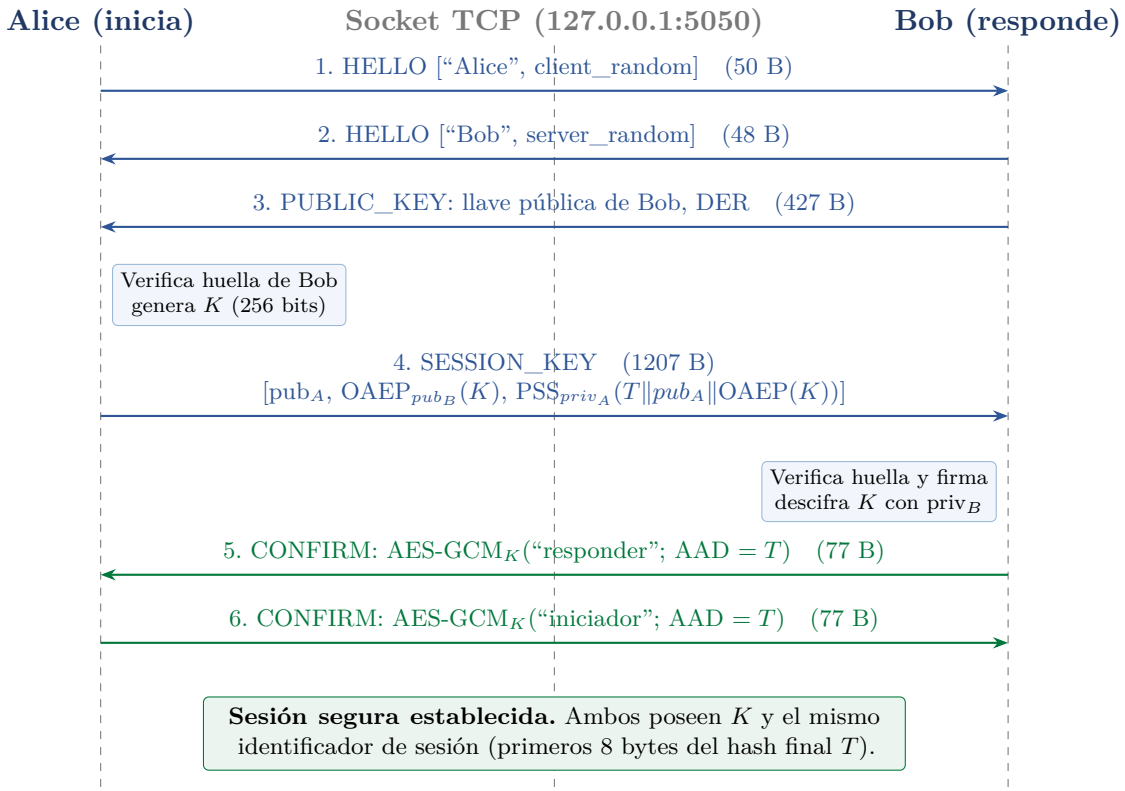


Figura 4.1: Handshake de SecureChat con los mensajes reales y su tamaño total en el cable (incluye el prefijo de longitud de 4 bytes). T es el hash SHA-256 acumulado de los mensajes anteriores (transcript).

4.1. Mensajes del handshake

Cada mensaje es una trama con 1 byte de tipo seguido del cuerpo. Los cuerpos con varios valores binarios se codifican como una lista de campos, cada uno precedido por su largo en 4 bytes (big-endian).

Tabla 4.1: Mensajes del handshake (tamaños con llaves RSA-3072)

#	Emisor	Tipo (byte)	Contenido y protección	Bytes
1	Alice	HELLO (1)	Nombre y *client_random* de 32 bytes. Público.	50
2	Bob	HELLO (1)	Nombre y *server_random* de 32 bytes. Público.	48
3	Bob	PUBLIC_KEY (4)	Llave pública de Bob en DER (422 bytes). Alice la valida con la huella registrada.	427
4	Alice	SESSION_KEY (5)	Llave pública de Alice (422 B); K cifrada con RSA-OAEP (384 B); firma RSA-PSS de Alice sobre $T \| pub_A \| \text{OAEP}(K)$ (384 B).	1207
5	Bob	CONFIRM (6)	Nonce de 12 bytes y la etiqueta “responder” cifrada con AES-256-GCM usando K y AAD $= T$ (incluye tag de 16 bytes).	77
6	Alice	CONFIRM (6)	Igual, con la etiqueta “iniciador”.	77
—	Cualquiera	ABORT (7)	Motivo del fallo en texto. Se envía antes de cerrar la conexión.	variable

4.2. Generación y protección de la clave de sesión

Alice genera K (apartado 3.6) y la cifra con la llave pública de Bob mediante RSA-OAEP (SHA-256, MGF1 con SHA-256); solo la llave privada de Bob puede recuperarla. Antes de enviar el mensaje, el programa comprueba que los 32 bytes de K no aparecen en los bytes que salen por el socket.

4.3. Transcript, firma y confirmación

Ambos lados calculan por separado un hash SHA-256 acumulado (T) de todos los mensajes del handshake, en orden. Este hash cumple tres funciones:

- **Firma de Alice.** Alice firma $T \| pub_A \| \text{OAEP}(K)$. Como T incluye los dos valores aleatorios y la llave pública de Bob, la firma solo es válida para *esta* sesión y no se puede reutilizar en otra.
- **Confirmación de clave.** Cada lado cifra una etiqueta fija con AES-GCM usando K y T como datos asociados (AAD). Si las claves o los transcripts difieren, la verificación del tag falla. Bob confirma primero (demuestra que descifró K) y Alice después.
- **Identificador de sesión.** Los primeros 8 bytes del hash final se muestran en ambas terminales; deben coincidir.

4.4. Autenticación mutua

Bob se autentica ante Alice porque su llave pública coincide con la huella registrada fuera de banda y porque su confirmación demuestra que posee la llave privada correspondiente. Alice se autentica ante Bob por su huella y por su firma RSA-PSS. Ninguna llave privada viaja por el socket: lo único que se serializa para enviarse son llaves *públicas* en formato DER.

4.5. Manejo de errores

Tabla 4.2: Fallos del handshake y reacción

Fallo	Reacción
Llave pública mal formada, no RSA o menor a 2048 bits	ABORT y cierre.
Huella recibida distinta de la registrada	ABORT (posible intermediario); se muestran ambas huellas.
Firma de Alice inválida	ABORT y cierre.
Descifrado RSA-OAEP fallido (`ValueError`)	ABORT y cierre.
Confirmación con tag inválido (`InvalidTag`)	ABORT y cierre.
Mensaje fuera de orden (por ejemplo, CHAT antes de HELLO)	ABORT y cierre.
Tiempo de espera agotado (15 s por mensaje)	ABORT y cierre.
Frase de paso incorrecta o falta la huella del par	No se abre la conexión.

En ningún caso se continúa sin una sesión establecida: el chat solo se habilita después de verificar ambas confirmaciones.

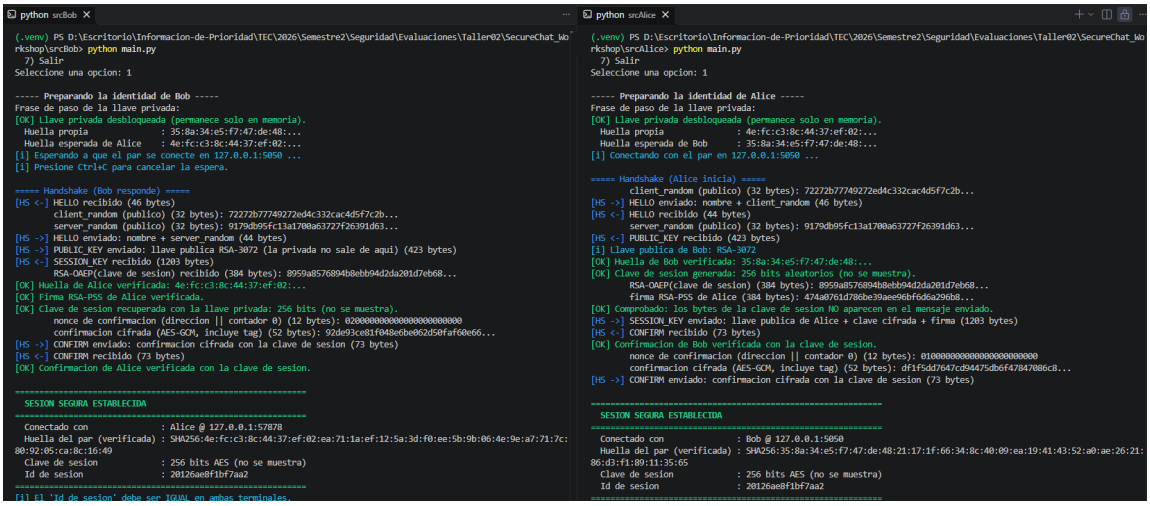


Figura 4.2: Ambas terminales muestran “SESIÓN SEGURA ESTABLECIDA” con el mismo identificador de sesión antes del primer mensaje.

5. Protección de los mensajes

Después del handshake, todo mensaje de usuario se cifra y autentica con AES-256-GCM antes de llegar al socket. No existe ninguna ruta en el código que escriba texto del usuario en claro. La despedida (BYE) también se cifra, para que un atacante no pueda inyectar un cierre falso.

5.1. AES-256-GCM

Tabla 5.1: Parámetros del cifrado simétrico autenticado

Parámetro	Valor
Algoritmo	AES-256-GCM (cifrado autenticado, AEAD (McGrew, 2008)), clase `AESGCM` de `cryptography`
Clave	La clave de sesión K de 256 bits del handshake
Nonce	96 bits = dirección (1 byte) ‖ contador (11 bytes, big-endian)
Tag	128 bits, concatenado al final del ciphertext
Datos asociados (AAD)	`"SecureChat-v1"` ‖ tipo (1 B) ‖ id de sesión (8 B) ‖ nonce (12 B)
Codificación del texto	UTF-8 (tildes y eñes)
Sobrecosto por mensaje	33 bytes (4 de longitud + 1 de tipo + 12 de nonce + 16 de tag)

5.2. Generación del nonce

Alice y Bob cifran con la *misma* clave K, por lo que cada dirección tiene su propio espacio de nonces. El primer byte indica la dirección (`01`: Alice → Bob, `02`: Bob → Alice) y los once restantes son un contador que crece en uno con cada mensaje.

- Dos nonces de direcciones distintas difieren siempre en el primer byte.
- Dos nonces de la misma dirección difieren siempre en el contador.
- El contador 0 de cada dirección se usa una sola vez, en la confirmación del handshake; el chat empieza en el contador 1.
- Si el contador llegara a $2^{88} - 1$, la sesión termina en lugar de reutilizar un nonce.

Por construcción, ningún nonce se repite con la misma clave. Se descartó el nonce aleatorio de 96 bits: con el volumen de un chat su riesgo de colisión sería despreciable, pero el contador ofrece la garantía por diseño y, además, sirve como número de secuencia.

Por qué el nonce no debe repetirse. En GCM, cifrar dos mensajes con la misma clave y el mismo nonce produce el mismo flujo de cifrado: el XOR de ambos ciphertexts revela el XOR de los plaintexts. Además, permite al atacante recuperar la subclave de autenticación y falsificar tags (Dworkin, 2007; Böck et al., 2016). El nonce, en cambio, puede viajar en claro: no es secreto, solo debe ser único.

5.3. Verificación en el receptor

El receptor procesa cada registro en este orden, y **solo** muestra el plaintext si todas las comprobaciones pasan:

1. **Formato:** el cuerpo mide al menos 28 bytes (nonce + tag).
2. **Dirección:** el primer byte del nonce debe ser el del par. Un mensaje propio devuelto por un atacante (reflejado) se rechaza.
3. **Secuencia:** el contador debe ser estrictamente mayor que el último aceptado de esa dirección.
4. **Autenticación:** `AESGCM.decrypt`; si lanza `InvalidTag`, el mensaje se rechaza y no se muestra nada.

El contador del receptor solo avanza después de un tag válido, de modo que un mensaje alterado no impide aceptar los mensajes legítimos siguientes.

5.4. Protección contra repetición

Como el número de secuencia viaja dentro del nonce y el nonce está autenticado, un atacante no puede cambiarlo sin invalidar el tag. Un registro capturado y reenviado llega con un número ya aceptado y se rechaza. La AAD incluye el tipo de mensaje y el id de sesión, por lo que un registro de chat no puede hacerse pasar por una despedida ni reutilizarse en otra sesión.

5.5. Evidencia en pantalla

El emisor ve su texto plano local y, debajo, lo que realmente sale por el socket: número de secuencia, tamaño de la trama y, en la vista detallada, nonce, ciphertext y tag en hexadecimal. El receptor ve lo recibido, luego `[TAG OK]` o `MENSAJE RECHAZADO` y, solo en el primer caso, el texto recuperado (figura 5.1). El hexadecimal es únicamente una forma de *mostrar* bytes; no protege nada.

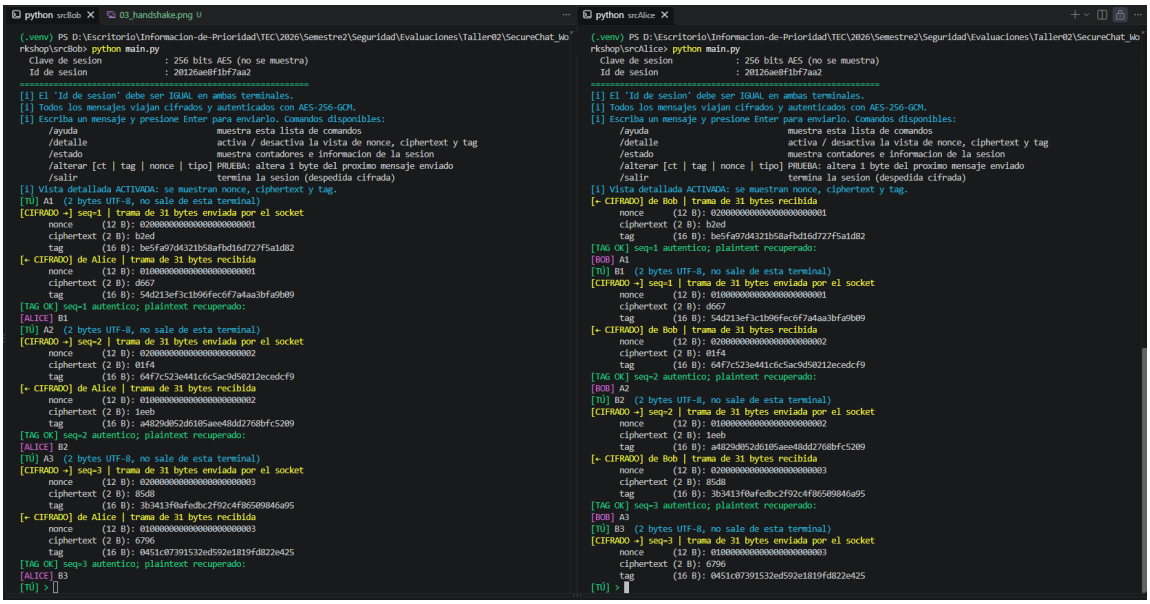


Figura 5.1: Emisor: plaintext local y ciphertext enviado. Receptor: ciphertext recibido, verificación del tag y plaintext recuperado.

6. Formato de los mensajes transmitidos

TCP entrega un flujo de bytes sin límites de mensaje, por lo que cada mensaje se delimita con un prefijo de longitud de 4 bytes. Las tramas de más de 64 KiB se rechazan antes de leer su contenido.

Tabla 6.1: Estructura de una trama de chat cifrada

Campo	Tamaño	Descripción
Longitud	4 bytes	Tamaño del resto de la trama (big-endian, `!I`). Visible.
Tipo	1 byte	2 = CHAT, 3 = BYE. Visible, pero autenticado en la AAD.
Nonce	12 bytes	Dirección (1 B) ‖ contador (11 B). Visible, pero autenticado.
Ciphertext	n bytes	Mensaje cifrado; mide lo mismo que el plaintext UTF-8.
Tag	16 bytes	Tag de autenticación de GCM.

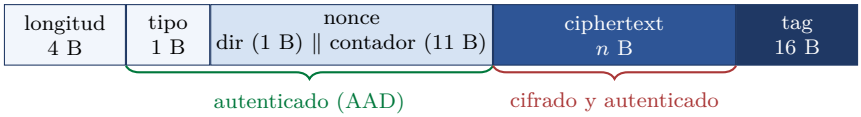


Figura 6.1: Trama de un mensaje de chat. Nada del texto del usuario viaja en claro.

Tabla 6.2: Tamaño real de las tramas en el cable

Mensaje	**Composición**	**Bytes**
CHAT con “hola 1” (6 B UTF-8)	$4+1+12+6+16$	39
CHAT con n bytes de texto	$4+1+12+n+16$	$33+n$
BYE (cuerpo vacío)	$4+1+12+0+16$	33
Trama máxima aceptada	prefijo + 64 KiB	65 540

El tamaño del ciphertext es igual al del plaintext, por lo que un observador puede deducir la longitud aproximada de cada mensaje (apartado 9.3).

Codificación no es cifrado. La representación en hexadecimal se usa solo para mostrar bytes en la consola. No ofrece confidencialidad: cualquiera puede revertirla sin ninguna clave. SecureChat no usa Base64 en ninguna parte y por el socket viajan bytes binarios.

7. Conversación bidireccional y cierre de sesión

7.1. Concurrencia y consola

Durante el chat trabajan dos hilos en cada instancia: el hilo principal lee el teclado, cifra y envía; un hilo receptor espera tramas, verifica y muestra. Así cualquiera de los dos participantes puede escribir en cualquier momento.

El prompt del chat no usa `input()`: en una consola de Windows lee tecla por tecla con `msvcrt`. Esto permite dos cosas: (1) si llega un mensaje mientras el usuario escribe, se imprime el mensaje y se vuelve a dibujar el prompt *con el texto ya escrito*; y (2) cuando el par cierra la sesión, el prompt deja de esperar y la instancia vuelve sola al menú. Toda impresión pasa por un único candado, de modo que las líneas de dos mensajes nunca se mezclan.

7.2. Presentación en consola

Tabla 7.1: Distinción visual en la consola

Etiqueta	Significado
`[TÚ] >`	Prompt del usuario local
`[TÚ] texto`	Mensaje local en texto plano (no sale de la terminal)
`[CIFRADO →]`	Trama enviada: número de secuencia, tamaño y, en vista detallada, nonce, ciphertext y tag
`[← CIFRADO]`	Trama recibida, antes de cualquier verificación
`[TAG OK]`	El tag de AES-GCM se verificó correctamente
`[ALICE] / [BOB]`	Texto recuperado, mostrado solo después de verificar el tag
`[!]`	Errores, advertencias y mensajes rechazados
`[i]`	Información

7.3. Comandos locales

Las líneas que empiezan con `/` son comandos locales y **nunca** se envían al par.

Tabla 7.2: Comandos disponibles durante el chat

Comando	Acción
`/ayuda`	Lista los comandos.
`/detalle`	Activa o desactiva la vista de nonce, ciphertext y tag (independiente en cada instancia).
`/estado`	Muestra el id de sesión, los mensajes enviados, recibidos y rechazados, y el último número de secuencia de cada dirección.
`/alterar [ct|tag|nonce|tipo]`	Prueba de integridad: el próximo mensaje sale con un byte alterado (apartado 8).
`/salir`	Envía una despedida cifrada y cierra la sesión.



Figura 7.1: Conversación intercalada con tres mensajes de Alice y tres de Bob.

7.4. Cierre controlado

`/salir` (o Ctrl+C) envía un mensaje BYE cifrado con AES-GCM. El otro extremo lo verifica como cualquier otro registro; si es auténtico, muestra que el par cerró la sesión y ambos vuelven al menú. Un BYE falso o alterado se rechaza, por lo que un atacante no puede cortar la conversación inyectando una despedida. Si el par desaparece sin despedirse, la desconexión se detecta y se informa. Al terminar, cada lado muestra un resumen con el motivo del cierre y los contadores de mensajes.

Borrado de la clave de sesión. Al cerrar, se eliminan todas las referencias del programa a la clave de sesión (`SecureChannel.close()` y `SessionMaterial.forget_session_key()`). Sin embargo, Python no permite garantizar que los bytes desaparezcan de la memoria: los objetos `bytes` son inmutables, el recolector de basura decide cuándo liberarlos y OpenSSL mantiene su propia copia interna. Lo que sí se garantiza es que ningún objeto de SecureChat puede volver a usar la clave.

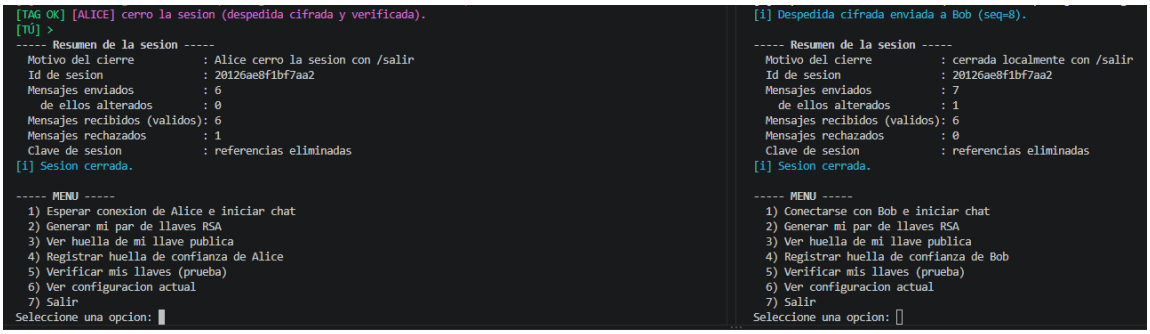


Figura 7.2: Cierre con `/salir`: despedida cifrada verificada y resumen de la sesión en ambas terminales.

8. Detección de manipulación

Cifrar no basta: un cifrado sin autenticación permitiría cambiar bits del ciphertext y que el receptor aceptara un mensaje distinto sin notarlo. Esta sección demuestra que el tag de GCM detecta cualquier alteración antes de que el receptor acepte el mensaje.

8.1. Mecanismo de la prueba controlada

Decisión. La alteración se hace dentro del emisor con el comando `/alterar`, activado a propósito por el usuario. El siguiente mensaje se cifra normalmente y, *después* de cifrar y *antes* de entregarlo al socket, la clase `RecordTamperer` cambia exactamente un byte del registro. El efecto en el receptor es idéntico al de una modificación en tránsito, porque el receptor solo ve los bytes que llegan. Se prefirió a un programa intermedio porque no exige cambiar puertos ni ejecutar un tercer proceso, y permite elegir con precisión qué campo se altera.

El byte se cambia en la carga útil de la trama; el prefijo de longitud nunca se toca, de modo que la trama llega completa y del mismo tamaño. Así se comprueba la criptografía y no el framing.

Tabla 8.1: Campos que se pueden alterar con `/alterar`

Opción	Byte alterado	Resultado esperado en el receptor
`ct` (por defecto)	Un byte en la mitad del ciphertext (XOR `0x01`)	`InvalidTag`: "MENSAJE RECHAZADO: autenticación fallida".
`tag`	Último byte del tag	`InvalidTag`.
`nonce`	Byte más significativo del contador	El contador crece y pasa la comprobación de secuencia, pero el nonce está autenticado: `InvalidTag`.
`tipo`	Byte de tipo CHAT (2) → BYE (3)	El receptor lo trata como despedida, pero el tipo forma parte de la AAD: `InvalidTag` y la sesión no se cierra.

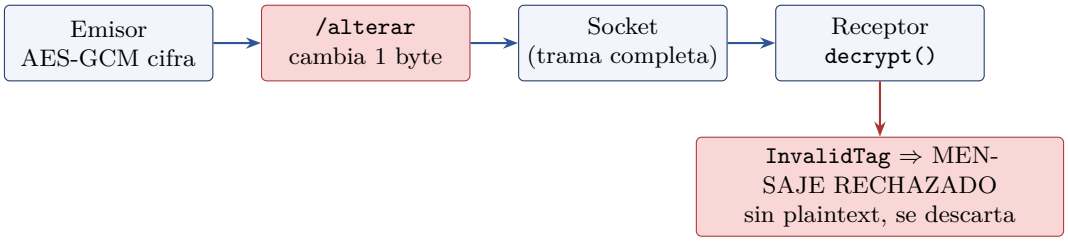


Figura 8.1: Prueba controlada de manipulación.

8.2. Resultado

En las cuatro variantes el receptor muestra `[← CIFRADO]` (los bytes llegaron), luego `[!] MENSAJE RECHAZADO: autenticación fallida` con el motivo, y **no imprime ningún plaintext**. Como el contador del receptor solo avanza después de un tag válido, el siguiente mensaje legítimo se acepta con normalidad (figuras 8.2 y 8.3).

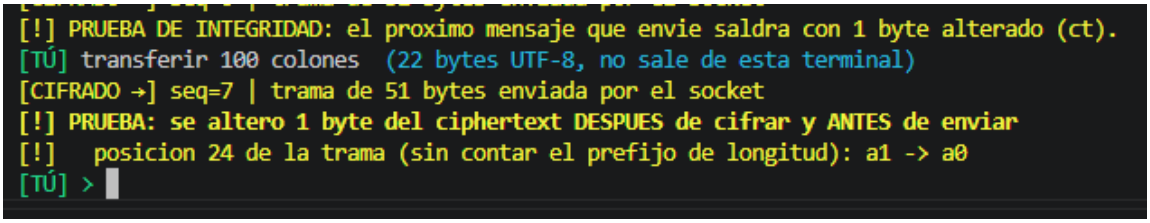


Figura 8.2: Emisor: `/alterar`, el mensaje cifrado y el byte cambiado (posición, valor original y valor alterado).

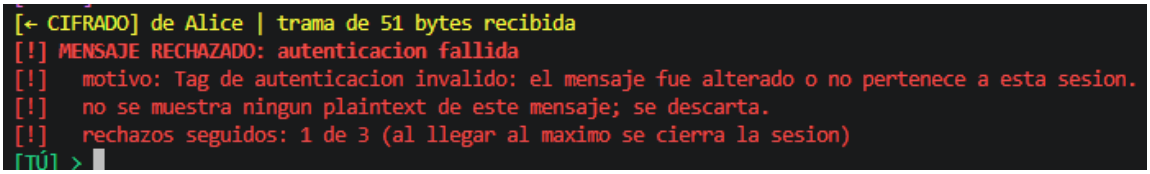


Figura 8.3: Receptor: el mensaje alterado se rechaza sin mostrar su contenido y el siguiente mensaje legítimo se acepta.

8.3. Política después de un rechazo

Decisión. Un mensaje rechazado se **descarta** y la sesión continúa, pero si se rechazan **3 mensajes seguidos** (`MAX_CONSECUTIVE_REJECTIONS`) el canal se considera comprometido y la sesión se cierra sin enviar despedida. Cualquier mensaje auténtico reinicia la cuenta.

La justificación es doble. Un rechazo aislado no compromete nada: el mensaje no se muestra, la ventana de secuencia no avanza y la clave sigue siendo secreta; cerrar de inmediato regalaría a un atacante un modo trivial de cortar la conversación. Pero varios rechazos seguidos indican que alguien manipula activamente el canal; en ese caso conviene dejar de confiar en él y establecer una sesión nueva con una clave nueva. El contador de rechazos se muestra en cada aviso y en `/estado`.

8.4. Pruebas automatizadas

Tabla 8.2: Pruebas automatizadas (`python -m pytest tests -v`)

Archivo	Pruebas	Qué comprueba
`test_secure_channel.py`	13	Plaintext ausente en los bytes enviados, 10 000 nonces sin repetición, direcciones disjuntas, cualquier byte alterado rechazado, repetición, reflexión, cambio de tipo, otra sesión y agotamiento del contador.
`test_console_input.py`	9	Prompt del chat: Backspace, teclas especiales, tildes, Ctrl+C, fin de sesión y texto preservado al llegar un mensaje.
`test_tampering.py`	19	Un byte alterado en ciphertext, tag, nonce o AAD produce `InvalidTag` (con `AESGCM` directo y con `SecureChannel`); `/alterar` cambia exactamente un byte sin cambiar el tamaño; repetición de una trama capturada; aceptación del siguiente mensaje legítimo; política de rechazos.
Total	**41**	

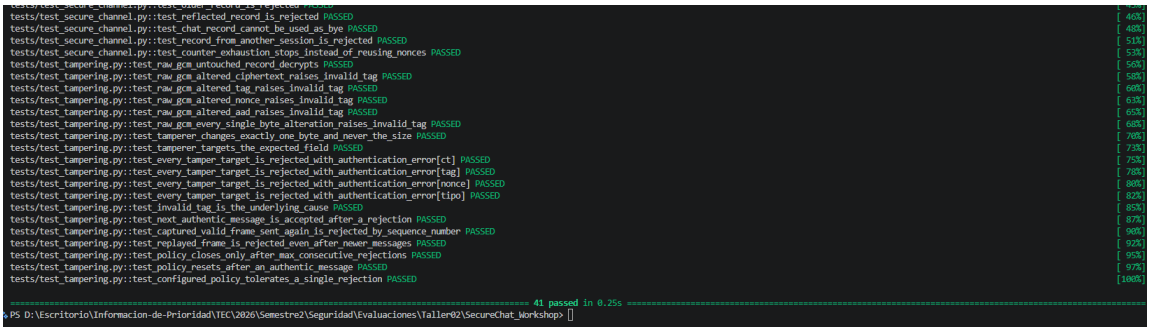


Figura 8.4: Las 41 pruebas automáticas aprobadas en Windows.

9. Análisis de seguridad

9.1. Modelo de adversario

Se considera el adversario de Dolev-Yao (Dolev and Yao, 1983): controla la red, puede leer, retener, modificar, repetir o inyectar cualquier byte, pero no puede romper los algoritmos

criptográficos ni acceder a la memoria o a los archivos de los participantes.

Tabla 9.1: Ataques considerados y defensa

Ataque	**Defensa en SecureChat**
Escucha pasiva del socket	Solo viaja ciphertext; K viaja cifrada con RSA-OAEP.
Sustitución de llaves públicas	Huellas SHA-256 verificadas fuera de banda; el handshake se aborta si no coinciden.
Suplantación de Alice	Firma RSA-PSS sobre el transcript de la sesión.
Suplantación de Bob	Solo quien posee la privada de Bob puede descifrar K y confirmar.
Modificación de un mensaje	Tag de GCM: `InvalidTag` y rechazo (apartado 8).
Repetición de un mensaje	Número de secuencia estrictamente creciente dentro del nonce autenticado.
Reflexión (devolver un mensaje)	Byte de dirección en el nonce.
Reutilizar un registro en otra sesión	El id de sesión forma parte de la AAD.
Despedida falsa para cortar el chat	El BYE también está cifrado y autenticado.
Repetición de un handshake grabado	Valores aleatorios nuevos en cada HELLO; la firma y la confirmación dependen del transcript.
Trama gigante (agotar memoria)	Límite de 64 KiB verificado antes de leer la carga útil.

9.2. Cumplimiento de los requisitos mínimos

Tabla 9.2: Requisitos mínimos del taller y su cumplimiento

	Requisito	**Cómo se cumple**	**Evidencia**
✓	Python 3 y sockets TCP/IP	Módulo `socket` (`AF_INET`, `SOCK_STREAM`) en `tcp_server.py` y `tcp_client.py`; framing en `framed_connection.py`.	figura 4.2
✓	Biblioteca criptográfica reconocida	`cryptography` (`requirements.txt`); no se implementa ninguna primitiva a mano.	`requirements.txt`
✓	Handshake basado en criptografía asimétrica	RSA-OAEP protege K; RSA-PSS y huellas autentican a ambos (apartado 4).	figuras 4.1 y 4.2
✓	Clave de sesión dinámica, no fija en el código	`generate_key` de `AESGCM` en cada conexión; la búsqueda de literales de llaves en el código no encontró ninguno.	apartado 3.6; id de sesión distinto en cada conexión
✓	Ninguna llave privada viaja por el socket	Solo se serializan llaves públicas (DER) para enviar; la privada solo se escribe cifrada en `keys/`.	cuadro 4.1
✓	Mensajes cifrados con esquema autenticado	AES-256-GCM en `secure_channel.py` para CHAT y BYE; no hay ruta de envío en claro.	figura 5.1; prueba automática de plaintext ausente
✓	Nonce correcto y no repetido	Dirección ‖ contador; cierre si se agota (apartado 5.2).	Prueba de 10 000 nonces

Continúa en la página siguiente

Tabla 9.2 – viene de la página anterior

	Requisito	Cómo se cumple	Evidencia
✓	Rechazo de tags inválidos	`InvalidTag` ⇒ MENSAJE RECHAZADO sin plaintext.	figura 8.3; `test_tampering.py`
✓	Evidencia de plaintext local, ciphertext enviado y recibido, y plaintext recuperado	Etiquetas `[TÚ]`, `[CIFRADO →]`, `[← CIFRADO]`, `[TAG OK]` y `[ALICE]`/`[BOB]`.	figura 5.1
×	Base64 o hexadecimal no sustituyen al cifrado	El hexadecimal solo se usa en `console_view.py` para mostrar; Base64 no se usa (búsqueda sin resultados).	apartado 6
×	No se usa ECB	Búsqueda de `ecb` en todo el código: sin resultados; el único modo simétrico es GCM.	apartado 5.1

Las verificaciones de código se ejecutaron desde la raíz del repositorio:

```
git grep -n -i "ecb"                # sin resultados
git grep -n -i "base64"             # sin resultados
git grep -n "generate_key"          # solo en alice_handshake.py
git grep -n "private_bytes"         # solo en key_store.py (escritura cifrada)
git grep -n "send_frame("           # handshake, CHAT/BYE cifrados y ABORT
git ls-files | Select-String "\.pem"      # sin resultados
```

Código 9.1: Verificaciones de código del Paso 7

9.3. Limitaciones conocidas

- **Confianza inicial en las huellas.** La protección contra un intermediario depende de que las huellas se hayan comparado correctamente al registrarlas.
- **Sin secreto hacia adelante.** K se transporta con RSA: quien obtenga en el futuro la llave privada de Bob podría descifrar sesiones grabadas. Un intercambio efímero (por ejemplo, ECDHE) firmado con RSA lo resolvería.
- **Análisis de tráfico.** Los tamaños, horas y puertos son visibles; el tamaño del ciphertext revela la longitud del mensaje.
- **Memoria.** En Python no se puede garantizar el borrado de secretos de la memoria (apartado 7.4).
- **Equipo local.** Quien controle la computadora (por ejemplo, un *keylogger*) ve los mensajes antes de cifrarlos; el protocolo protege el canal, no los extremos.

10. Preguntas de cierre

1. **¿Qué problema resuelve la criptografía asimétrica durante el handshake?**
   Permite acordar una clave secreta sin haber compartido antes ningún secreto. Alice cifra la clave de sesión con la llave pública de Bob y solo Bob puede recuperarla con su privada. Además autentica a los participantes: la huella de cada llave pública y la firma de Alice impiden que un tercero se haga pasar por alguno de ellos.

2. **¿Por qué la conversación utiliza criptografía simétrica después del handshake?**
   Porque es mucho más rápida y no tiene límite práctico de tamaño. RSA solo cifra unos cientos

de bytes por operación y es miles de veces más costoso; AES-GCM cifra y autentica cada mensaje de cualquier tamaño con poco costo y solo agrega 33 bytes por trama.

3. **¿Qué información puede observar un tercero que capture los paquetes de la conversación?**
   Las direcciones y puertos, las horas y el tamaño de cada trama (y por tanto la longitud aproximada de cada mensaje), los nombres y valores aleatorios del HELLO, las llaves públicas, la clave de sesión *cifrada*, la firma, y de cada mensaje el tipo, el nonce, el ciphertext y el tag. No puede ver el texto de los mensajes ni la clave de sesión.

4. **¿Por qué el nonce puede viajar junto al ciphertext y por qué no debe repetirse?**
   Puede viajar en claro porque no es secreto: su función es que cada cifrado sea distinto, y sin la clave no sirve para nada. No debe repetirse porque, con la misma clave y el mismo nonce, GCM produce el mismo flujo de cifrado: el XOR de dos ciphertexts revela el XOR de los plaintexts, y se pueden falsificar tags. SecureChat lo evita con un contador por dirección.

5. **¿Qué función cumple el authentication tag?**
   Garantiza integridad y autenticidad: es un valor de 128 bits que depende de la clave, del nonce, del ciphertext y de los datos asociados. Si cambia un solo bit de cualquiera de ellos, la verificación falla y el receptor rechaza el mensaje sin mostrar su contenido. Sin la clave, un atacante no puede calcular un tag válido.

6. **¿Qué ocurriría si la clave de sesión se transmitiera sin protección?**
   Cualquiera que capturara el handshake la obtendría y podría descifrar todos los mensajes de la sesión, e incluso cifrar mensajes falsos con tags válidos. AES-GCM seguiría funcionando, pero ya no protegería nada: la seguridad depende por completo de que la clave sea secreta.

7. **¿Qué ocurriría si un atacante obtiene una de las claves privadas?**
   Con la privada de Bob podría descifrar la clave de sesión de cualquier handshake grabado (pasado o futuro) y leer esas conversaciones, además de hacerse pasar por Bob. Con la de Alice podría firmar como ella y suplantarla ante Bob. En ambos casos hay que generar llaves nuevas y registrar las nuevas huellas. Por eso las privadas se guardan cifradas con una frase de paso y nunca salen de la carpeta de su dueño.

11. Evidencias de la demostración

Tabla 11.1: Resultados esperados por el enunciado y evidencia

#	Resultado esperado	Evidencia
1	Dos instancias ejecutándose simultáneamente	figura 4.2
2	Handshake completado antes del primer mensaje	figura 4.2
3	Tres mensajes de Alice y tres de Bob	figura 7.1
4	Ciphertext que cruza la conexión	figura 5.1
5	Recuperación del plaintext en el receptor	figura 5.1
6	Ciphertext alterado detectado y rechazado	figuras 8.2 y 8.3

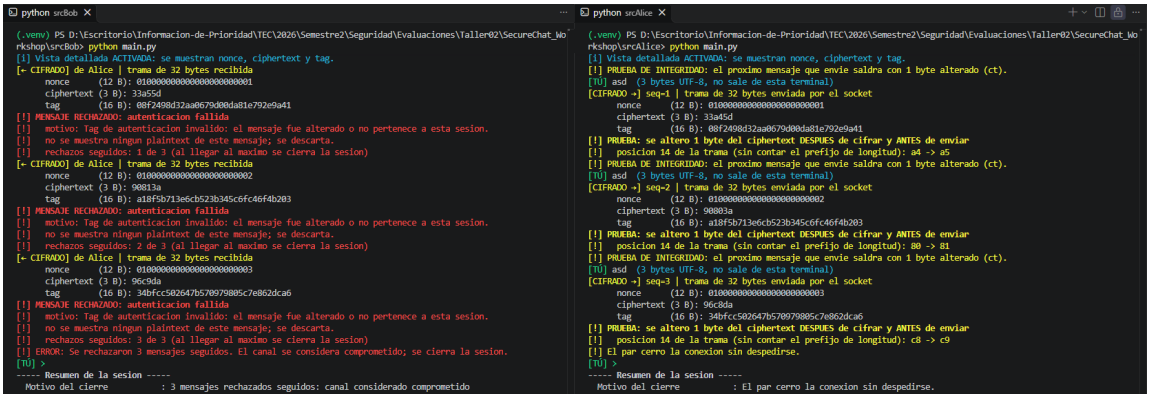


Figura 11.1: Política tras rechazos: al tercer mensaje rechazado seguido la sesión se cierra.

Video de demostración: https://youtu.be/uzH1AXT1bYw

Referencias

Barker, E. (2020). Recommendation for key management: Part 1 – general. NIST Special Publication 800-57 Part 1 Rev. 5, National Institute of Standards and Technology.

Bleichenbacher, D. (1998). Chosen ciphertext attacks against protocols based on the RSA encryption standard PKCS #1. In *Advances in Cryptology – CRYPTO '98*, volume 1462 of *Lecture Notes in Computer Science*, pages 1–12. Springer.

Böck, H., Zauner, A., Devlin, S., Somorovsky, J., and Jovanovic, P. (2016). Nonce-disrespecting adversaries: Practical forgery attacks on GCM in TLS. In *10th USENIX Workshop on Offensive Technologies (WOOT 16)*. USENIX Association.

Dolev, D. and Yao, A. C. (1983). On the security of public key protocols. *IEEE Transactions on Information Theory*, 29(2):198–208.

Dworkin, M. (2007). Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC. NIST Special Publication 800-38D, National Institute of Standards and Technology.

McGrew, D. (2008). An interface and algorithms for authenticated encryption. RFC 5116, Internet Engineering Task Force.

Moriarty, K., Kaliski, B., Jonsson, J., and Rusch, A. (2016). PKCS #1: RSA cryptography specifications version 2.2. RFC 8017, Internet Engineering Task Force.

Python Cryptographic Authority (2026). cryptography: documentación oficial (AESGCM, RSA, OAEP, PSS y serialización). https://cryptography.io/en/latest/. Consultado en octubre de 2026.

Python Software Foundation (2026). socket – low-level networking interface. https://docs.python.org/3/library/socket.html. Consultado en octubre de 2026.

Vargas Rivera, A. F. (2026). Material del curso CE-1115 Seguridad de la Información: fundamentos, tríada CIA, ataques activos y pasivos, autenticación y protección del canal. Tecnológico de Costa Rica, Escuela de Ingeniería en Computadores. Notas de clase, II Semestre 2026.

A. Ejecución y estructura del repositorio

```
SecureChat_Workshop/
|-- srcAlice/      main.py, config.py, tcp_client.py, alice_handshake.py
|   |-- .venv/     entorno virtual de Alice (no se versiona)
|   `-- keys/      llaves de Alice
|-- srcBob/        main.py, config.py, tcp_server.py, bob_handshake.py
|   |-- .venv/     entorno virtual de Bob (no se versiona)
|   `-- keys/      llaves de Bob
|-- srcShared/     codigo comun a ambos participantes
|-- tests/         test_secure_channel.py, test_console_input.py, test_tampering.py
|-- DescripcionDelProtocolo/ este documento, referencias.bib e img/
|-- requirements.txt, CurrentCommands.txt, README.md
```

Código A.1: Estructura del repositorio

```
py -3 -m venv srcBob\.venv
srcBob\.venv\Scripts\python.exe -m pip install -r requirements.txt
py -3 -m venv srcAlice\.venv
srcAlice\.venv\Scripts\python.exe -m pip install -r requirements.txt

# Terminal 1 (primero)            # Terminal 2
srcBob\.venv\Scripts\python.exe srcBob\main.py
srcAlice\.venv\Scripts\python.exe srcAlice\main.py

# Pruebas automaticas
srcBob\.venv\Scripts\python.exe -m pytest tests -v
```

Código A.2: Instalación y ejecución en Windows (PowerShell, desde la raíz)

La primera vez, cada participante genera sus llaves (opción 2), muestra su huella (opción 3) y registra la del otro (opción 4). Después, Bob elige la opción 1 y espera; Alice elige la opción 1 y se conecta. El README contiene el guion completo de la demostración.

B. Declaración de uso de IA

Durante el taller se utilizó un asistente de inteligencia artificial (Claude, de Anthropic) como apoyo para planificar los pasos del desarrollo, proponer la estructura de clases, redactar borradores del código, de las pruebas automáticas y de esta documentación, y explicar conceptos del protocolo.

Los integrantes revisaron todo el código y el texto generados, tomaron las decisiones de diseño (tamaño de llave, modelo de distribución de llaves, estrategia de nonce y política tras un rechazo), ejecutaron cada paso en Windows, corrieron las 41 pruebas automáticas y verificaron manualmente la conversación y la prueba de manipulación antes de dar cada paso por terminado. Ambos integrantes comprenden el flujo completo del protocolo y asumen la responsabilidad del trabajo entregado.